ROS2 Joint Control: Extension Python Scripting

Learning Objectives

In this tutorial, you interact with a manipulator, the Franka Emika Panda Robot. You:

- Add a ROS2 Joint State publisher and subscriber in OmniGraph
- Add a ROS2 Joint State publisher and subscriber using menu shortcut
- Add a ROS2 Joint State publisher and subscriber using the OmniGraph Python API using the script editor
- Learn about the `isaac:nameOverride` prim attribute

Getting Started

Prerequisite

- Completed Workflows to understand the Extension Workflow.
- Appropriate `ros2_ws` is sourced in the terminal that will be running Python scripts.
- `FASTRTPS_DEFAULT_PROFILES_FILE` environment variable is set prior to launching Isaac Sim and the ROS2 bridge is enabled.

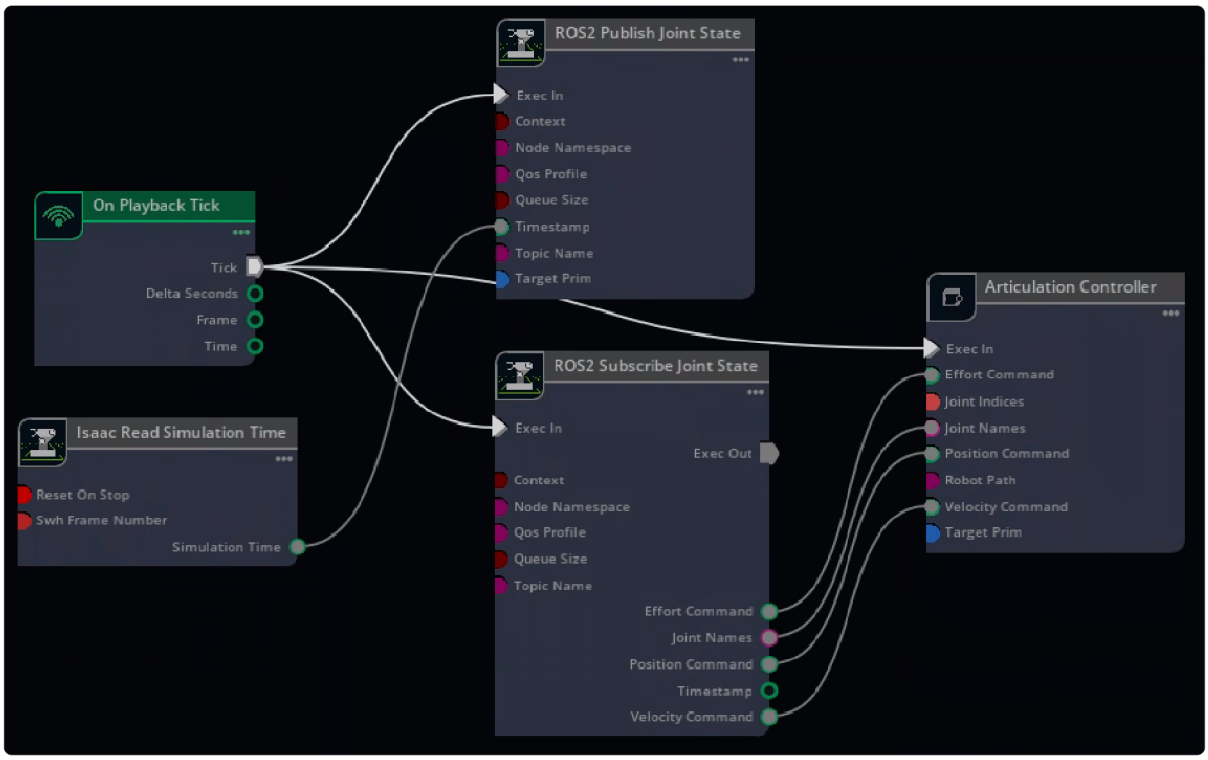
Add Joint States in UI

1. Open the asset, which can be found by going to the Isaac Sim Content browser **Isaac Sim>Robots>FrankaRobotics>FrankaPanda>franka.usd**.
2. Go to **Window > Graph Editors > Action Graph** to create an Action graph.
3. Add the following OmniGraph nodes into the Action graph:

> - **On Playback Tick** node to execute other graph nodes every simulation frame.
> - **Isaac Read Simulation Time** node to retrieve current simulation time.
> - **ROS2 Publish Joint State** node to publish ROS2 Joint States to the `/joint_states` topic.
> - **ROS2 Subscribe Joint State** node to subscribe to ROS2 Joint States from the `/joint_command` topic.

- **Articulation Controller** node to move the robot articulation according to commands received from the subscriber node.

4. Select *ROS2 Publish Joint State* node and add the */panda* robot articulation to the *targetPrim*.
5. Select *Articulation Controller* node, indicate the robot articulation you want to move by adding */panda* to the *targetPrim*, or typing */panda* in the *robotPath* field.
6. Connect the Tick output of the *On Playback Tick* node to the Execution input of the *ROS2 Publish Joint State*, *ROS2 Subscribe JointState*, and *Articulation Controller* nodes.
7. Connect the Simulation Time output of the *Isaac Read Simulation Time* node to the Timestamp input of the *ROS2 Publish Joint State* node. Setup other connections between nodes as shown in the image below:



8. Press **Play** to start publishing joint states to the `/joint_states` topic and subscribing commands on the `/joint_command` topic.
9. To test out the ROS2 bridge, use the provided Python script to publish joint commands to the robot. In a ROS2-sourced terminal:

```
ros2 run isaac_tutorials ros2_publisher.py
```

10. While the robot is moving, open a ROS2-sourced terminal and check the joint state ROS 2 topic by running:

```
ros2 topic echo /joint_states
```

> **Note**
>
> Articulation Root describes the beginning of an articulation tree, a collection of links and joints that makes up the robot in simulation. For fixed base robots like the franka, articulation root is specified at its root joint to world, and for move-able objects, the articulation root is specified at the rigid body with the deepest tree, typically the torso or chassis_link.

Graph Shortcut

We provide a menu shortcut to build Joint State Publisher and Subscriber graphs with just a few clicks. Go to **Tools > Robotics > ROS 2 OmniGraphs > JointStates**. If you don't observe any ROS2 graphs listed, you need to enable the ROS2 bridge. A popup box will appear asking for the parameters needed to populate the graphs. You must provide the Graph Path, Node Namespace if there is one needed, and the prim that contains the Articulation Root API. If you are using the subscriber, you also have the option to add the Articulation Controller node needed to move the robot.

Add Joint States in Extension

The same action done using the UI can also be done using a Python script. More details regarding the different workflows of using NVIDIA Isaac Sim can be found Workflows.

1. Open the asset, which can be found by going to the Isaac Sim Content browser and clicking **Isaac Sim>Robots>FrankaRobotics>FrankaPanda>franka.usd**.
2. Open Script Editor in **Window > Script Editor** and copy paste the following code into it. This is the equivalent to Steps 2-7 of the previous section. If the robot appears other than */panda* on the stage tree, make sure to match the Articulation Controller and Publish Joint State nodes' targets to the robot's prim path (line 29 and 30).

```
import omni.graph.core as og

og.Controller.edit(
    {"graph_path": "/ActionGraph", "evaluator_name": "execution"},
    {
        og.Controller.Keys.CREATE_NODES: [
            ("OnPlaybackTick", "omni.graph.action.OnPlaybackTick"),
            ("PublishJointState", "isaacsim.ros2.bridge.ROS2PublishJoint
            ("SubscribeJointState", "isaacsim.ros2.bridge.ROS2Subscribe
            ("ArticulationController", "isaacsim.core.nodes.IsaacArticu
            ("ReadSimTime", "isaacsim.core.nodes.IsaacReadSimulationTime
        ],
        og.Controller.Keys.CONNECT: [
            ("OnPlaybackTick.outputs:tick", "PublishJointState.inputs:ex
            ("OnPlaybackTick.outputs:tick", "SubscribeJointState.inputs
            ("OnPlaybackTick.outputs:tick", "ArticulationController.inpu
            ("ReadSimTime.outputs:simulationTime", "PublishJointState.in
            ("SubscribeJointState.outputs:jointNames", "ArticulationCont
            ("SubscribeJointState.outputs:positionCommand", "Articulatic
            ("SubscribeJointState.outputs:velocityCommand", "Articulatic
            ("SubscribeJointState.outputs:effortCommand", "ArticulationC
        ],
        og.Controller.Keys.SET_VALUES: [
            # Providing path to /panda robot to Articulation Controller
            # Providing the robot path is equivalent to setting the targ
            # ("ArticulationController.inputs:usePath", True),      # if
            ("ArticulationController.inputs:robotPath", "/panda"),
            ("PublishJointState.inputs:targetPrim", "/panda"),
        ],
    },
)
```

3. Press **Run** in the **Script Editor** and the Action Graph with all required nodes is added. You can find the corresponding ActionGraph in the Stage Tree.

> **Note**
>
> This script must only be run once. It is assuming there is no ActionGraph that already exists on stage. You can start a new stage to run it again.

4. Test out the ROS 2 bridge using the provided ROS 2 Python node to publish joint commands to the robot. In a ROS 2 sourced terminal, run the following command:

```
ros2 run isaac_tutorials ros2_publisher.py
```

5. Verify the joint state with a ROS 2 topic echo command while it's moving:

```
ros2 topic echo /joint_states
```

Position and Velocity Control Modes

The joint state subscriber supports position and velocity control. Each joint can only be controlled by a single mode at a time, but different joints on the same articulation tree can be controlled by different modes. Make sure each joint's stiffness and damping parameters are setup appropriately for the desired control mode (position control: stiffness >> damping, velocity control: stiffness = 0).

The snippet is an example of how to command a robot using both position and velocity controls by grouping joints that use the same mode into one message, and create two different messages for position control joints and velocity controlled joints. Separating them is for organization and potentially sending them at different rates.

```
import threading

import rclpy
from sensor_msgs.msg import JointState

rclpy.init()
node = rclpy.create_node("position_velocity_publisher")
pub = node.create_publisher(JointState, "joint_command", 10)

# Spin in a separate thread
thread = threading.Thread(target=rclpy.spin, args=(node,), daemon=True)
thread.start()

joint_state_position = JointState()
joint_state_velocity = JointState()

joint_state_position.name = ["joint1", "joint2", "joint3"]
joint_state_velocity.name = ["wheel_left_joint", "wheel_right_joint"]
joint_state_position.position = [0.2, 0.2, 0.2]
joint_state_velocity.velocity = [20.0, -20.0]

rate = node.create_rate(10)
try:
    while rclpy.ok():
        pub.publish(joint_state_position)
        pub.publish(joint_state_velocity)
        rate.sleep()
except KeyboardInterrupt:
```


Last updated on Jun 22, 2026.